

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**Advanced REST API Development
with Django**

Bachelor's Thesis

MICHAL ŠOLTIS

Brno, Spring 2023

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Advanced REST API Development with Django

Bachelor's Thesis

MICHAL ŠOLTIS

Advisor: Mgr. Luděk Bártek, Ph.D.

Department of Computer Systems and Communications

Brno, Spring 2023



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Michal Šoltis

Advisor: Mgr. Luděk Bártek, Ph.D.

Acknowledgements

I want to thank my family and friends for their support during my academic journey. I am grateful for their contributions and everything they have done for me. Their love has been a source of strength and inspiration. Without them, I could not reach this milestone in my life. Thank you once again.

Abstract

The aim of the thesis is to explore a high-level Python web framework Django and provides insight into the framework's capabilities, best practices, and standard patterns. Part of the thesis is the implementation of the REST API with Django, serving as a demo application. This demo application helps cover Django's advanced concepts alongside various valuable tools for REST API development in general.

Keywords

Python, Django, REST API, Backend, Web development

Contents

Introduction	1
1 Python Web Frameworks	2
1.1 Django	3
1.2 Flask	4
1.3 FastAPI	4
2 Django Basics	5
2.1 Project structure	5
2.2 Models	7
2.3 ORM	7
2.4 Database	8
2.5 Views	8
2.6 URLs	9
2.7 Admin interface	9
2.8 Development server	10
2.9 Commands summary	10
3 Technology Stack	11
3.1 REST API	11
3.2 REST API criteria	11
3.3 Django REST framework	12
3.4 Celery	12
4 Building a REST API	14
4.1 Use Case Diagram	15
4.2 Dependencies	16
4.3 ERD diagram	16
4.4 Serializers	18
4.5 View sets	20
4.6 Routers	22
4.7 Background tasks	23
4.8 Signals	24
5 Testing and Optimization	26

5.1	Automated Testing	26
5.1.1	Code quality	26
5.2	Performance Testing	26
5.2.1	Optimization	28
Conclusion		30
Bibliography		31
A An appendix		33

List of Figures

1.1	Python web frameworks popularity [3]	3
2.1	Django administration panel	10
4.1	Use Case Diagram	15
4.2	Entity Relationship Diagram	17
4.3	Django REST framework browsable API	23
4.4	REST API profile endpoint	24
4.5	REST API register endpoint	25
5.1	Locust performance test	27
5.2	Silk profiling dashboard	28

Introduction

The concept of REST or RESTful API has become a fundamental building block for software development. REST (REpresentational State Transfer) is a set of architectural constraints and provides a standardized set of guidelines for creating web services and efficient communication between client applications and server backends [1].

Python web framework Django provides a flexible platform for building web applications. It has gained popularity among developers due to its versatility and pragmatic design. The Django and REST API combination is a good starting point for backend development.

The primary goal of this thesis is to provide an overview of the current state of the Django framework and to demonstrate the development of REST API with Django using a practical and illustrative example. The final application is a simple sports platform that simulates matches in the leagues by randomly generating data. The frontend development is not part of this thesis.

Chapter 1 presents an overview and comparison of Python web frameworks. Chapter 2 contains the general development workflow with Django and its basic concepts. Chapter 3 summarizes the technology I used during development. Chapter 4 demonstrates the design and implementation process of the REST API with the Django REST framework. Finally, chapter 5 concludes the thesis with a short explanation of the testing and optimization process of the REST APIs.

In conclusion, this thesis contributes to a better understanding of Python web development using various tools possibly applicable to any Python project.

1 Python Web Frameworks

Python has become popular in web development, with numerous frameworks available for creating and managing web applications, web services, or websites. These frameworks offer various features and benefits, making Python a popular language for web development.

Using a framework to implement an idea may require following many conventions, standards, and rules. Some frameworks do not let developers implement their ideas in their way, meaning the frameworks force developers to use “prescribed architecture” in the projects. The solution might be a reverse strategy, choosing the project’s architecture first and then finding the appropriate framework for it [2].

This chapter summarizes and shows the differences among the three most popular Python web frameworks based on the JetBrains Python Developers Survey from 2021 [3]. These are Flask, Django, and FastAPI.

Each year, the survey reveals the current state of the language. Python developers submit multi-answer questions regarding various areas of Python. In 2020, over 23,000 Python developers and enthusiasts from almost 200 countries took this survey. Figure 1.1 shows the results of the question regarding the usage of web frameworks. Flask, Django, and FastAPI have a big lead over other frameworks.

Frameworks and Libraries

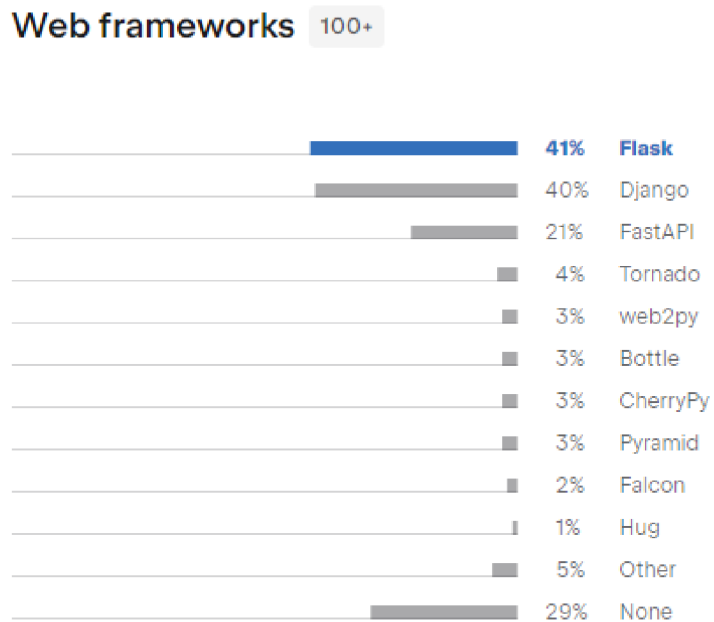


Figure 1.1: Python web frameworks popularity [3]

1.1 Django

Django is a free and open-source web framework prioritizing rapid development and clean, practical design. It provides high productivity and efficiency for web development. It has many built-in features that help developers build and compose robust applications with other previously written components. The official documentation of Django itself characterizes the framework as “the web framework for perfectionists with deadlines” [4].

Many big companies use Django, for example, Youtube, Instagram, Spotify, or Dropbox.

For more information about Django, visit the official documentation [4].

1.2 Flask

Flask is considered a micro web framework. In this context, micro refers to its simplicity. It is easier to learn than Django because Flask does not mandate specific dependencies or structure of the project. Instead, it allows developers to select their preferred tools and libraries.

Additionally, many community-supported extensions are available that simplify incorporating new features into the Flask project.

For more information about Flask, visit the official documentation [5].

1.3 FastAPI

FastAPI is a relatively new framework released at the end of 2018. It has a smaller community than Django or Flask.

The framework uses type hints as its foundation. FastAPI requires explicitly adding a type annotation to the API, unlike previously mentioned frameworks. The actual data validation and error handling are done automatically.

As the name suggests, the framework provides high performance and the ability to create APIs fast. It belongs among the top fastest Python web frameworks.

For more information about FastAPI, visit the official documentation [6].

2 Django Basics

This chapter gives an overview of how to use Django to start developing web applications.

Django is straightforward regarding conventions, standards, and the project's architecture. In this thesis, I often refer to these conventions and standards.

2.1 Project structure

The creation of a new Django project can be initiated through the use of the following command:

```
django-admin startproject <name>
```

Upon execution of this command, an initial directory will be generated, serving as the fundamental structure of the entire project. Outside this directory, Django also creates a file named `manage.py`.

The `manage.py` file is a wrapper for the `django-admin` command. Upon invoking the `manage.py` file, the `DJANGO_SETTINGS_MODULE` environment variable is automatically set to reference the `settings.py` module in the initial project directory. Thus, the following syntax can be implied to execute any command related to the Django project:

```
python manage.py <command>
```

The initial directory contains these files:

- `asgi.py`¹
- `settings.py`
- `urls.py`
- `wsgi.py`²

1. Asynchronous Server Gateway Interface

2. Web Server Gateway Interface

Every Django project is composed of various apps or app components. All apps must be registered in the list of `INSTALLED_APPS`, as shown below, in the `settings.py` module, which already contains some apps that every Django project includes by default. One of the benefits of using Django is that developers do not have to start their projects from scratch, but they can build the final project on these pre-installed apps.

Generally, every app component should be reusable and responsible for one thing. Before implementing a new project, redistributing functionality into multiple apps is a good practice. Hence, they are not dependent on each other, making them reusable for other projects. This process allows the creation of a cohesive and structured Django project with organized and distinct app components, with one core app that connects and manages the other app components. The idea is similar to modular design principles³.

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
]
```

The following command generates a new directory in the root folder of the project, which represents the new app:

```
python manage.py startapp <name>
```

The directory contains various files crucial for the app functionality, including a migrations directory.

The module `apps.py` contains a simple configuration of the app. Other filenames within the directory are self-descriptive, providing insight into their purpose within the app's architecture.

- migrations

3. For more information about modular design, visit – https://en.wikipedia.org/wiki/Modular_design

- `admin.py`
- `apps.py`
- `models.py`
- `tests.py`
- `urls.py`
- `views.py`

2.2 Models

Usually, the first step in every Django project is to figure out and design data models. All models of an app should be defined in the `models.py` module. Each model represents a single table in the database. The names of the tables start with the prefix of the corresponding name of the app.

The `id` attribute is added to all models by default but can be replaced with any attribute as a unique key.

2.3 ORM

Object-relational mapper (ORM) is a technique to map data between a relational database and an object-oriented programming language. In other words, it is an abstraction above the database.

Django has its own ORM, which allows performing CRUD (Create, Read, Update, Delete) operations on models, which are reflected in the database. However, Django also provides the flexibility to execute raw SQL queries when necessary, which allows developers to have greater control over the queries executed on the database. They are helpful when working with complex data models or queries that cannot be expressed using ORM.

In Django, all queries or query sets are lazy, which means that a query set is not evaluated and does not hit the database until the requested data are needed. After the first evaluation of the query set, Django stores the result in the so-called query cache. However, the entire query must be evaluated.

2.4 Database

Database settings are located in the `settings.py` module. Initially, the default SQL database engine for Django is the library SQLite.

SQLite is simple and fast and wraps the database into a single file.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Every app contains a migrations folder.

Migrations are used to create or update tables in the database. A migration file is a Python module located in the migrations folder of a corresponding app. It contains instructions to create tables and fields in the database. The file's name is made from the sequence number and auto-generated name, which can be manually changed for a more descriptive name. Django offers other commands to work with migrations, such as reverting and squashing. It is very similar to working with a versioning system. Each migration is like a commit.

The first command below creates initial migrations. These migrations are pushed to the database via the second command below. The command runs through all installed apps and pushes the latest changes to the database by executing instructions in the migration files. The first execution also pushes the latest changes from all pre-installed apps that the Django project contains by default.

```
python manage.py makemigrations
python manage.py migrate
```

2.5 Views

Django views are Python functions that take HTTP requests and return HTTP responses. These functions are so-called request handlers. By convention, each app's views are defined in the `views.py` module. The filename might be confusing since the word view can be associated

with something a user can see. In Django, for some reason, request handlers are called views.

2.6 URLs

Views or view functions are mapped to URLs. The URLs of each app should be defined in the `urls.py` module. Every app can have its own URL configuration. The initial project directory contains another `urls.py` module, the central URL configuration.

To map URLs to views in each app's corresponding `urls.py` module, Django looks for a special variable called `urlpatterns`. It is an array of URL pattern objects that map views to URLs. Also, this variable is located in the main `urls.py` module in the project directory and contains the URL configuration of all app components.

2.7 Admin interface

One of the notable features of Django is its admin interface, which developers can use as an internal management tool. It is an app that the initial project template contains by default. This interface enables admin users to manage the sources across the whole Django project.

In each app, the `admin.py` module holds customization of the admin panel of that app and the registration of models to make them manageable in the admin interface. A Figure 2.1 shows the front page of the admin site.

The following command creates a super user (admin user) with access to the admin interface.

```
python manage.py createsuperuser
```

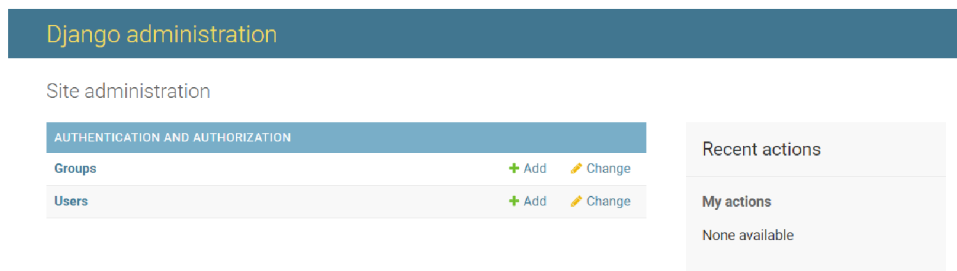


Figure 2.1: Django administration panel

2.8 Development server

The Django development server is a lightweight web server included in the Django web framework.

The server offers a valuable feature of automatically reloading the Python code for each incoming request, which allows to preview the changes in the applications in real time. However, some operations, such as adding files, may not trigger the automatically reload mechanism. In such cases, a manual server restart is necessary.

The following command initiates the development server. The development server initializes on the localhost IP address, with port 8000 being the default configuration. The web page shows a rocket with the congratulations label, indicating the successful server launch.

```
python manage.py runserver
```

2.9 Commands summary

```
django-admin startproject <name>
python manage.py startapp <name>
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

3 Technology Stack

This chapter provides an overview of the key technologies utilized in the demo project, including their theoretical basis.

3.1 REST API

REST (REpresentational State Transfer) is an architectural style for building web services. Unlike a protocol or standard, REST defines a set of constraints that API developers can follow to create scalable and flexible web services. When a client sends a request to a RESTful API, it receives a representation of the resource state in a format such as JSON, HTML, or plain text. JSON is the most commonly used among these formats, as it is language-agnostic, human-readable, and machine-understandable. By adhering to REST's principles, API developers can build web services that are easy to maintain, evolve, and integrate with other systems [1].

3.2 REST API criteria

For an API to be considered RESTful, it must meet the following criteria:

1. A client-server architecture with HTTP managing requests and resources are defined as separate entities.
2. A client-server architecture with HTTP managing requests and resources are defined as separate entities.
3. Stateless communication between client and server means each request must be self-contained and not depend on previous ones, and the server cannot store any client data.
4. The ability to cache data enhances client-server interactions and reduces network traffic.
5. A uniform interface between components, allowing information to be transmitted in a standardized format, which requires:

- (a) Resources must have a unique identifier and be separate from the representation sent to the client.
 - (b) The representation sent to the client should contain all the necessary information to allow it to modify the resources.
 - (c) Self-descriptive messages returned to the client should include enough information to specify how the client should handle them.
 - (d) Hypertext/hypermedia links should be available to help the client navigate to other available actions.
6. A layered system that organizes the server functions into hierarchies that are invisible to the client, such as security or load-balancing.
7. Code-on-demand is optional, which means the server can transmit executable code to the client upon request, allowing for more extensive client capabilities.

Meeting these criteria ensures that the API conforms to the RESTful architectural style and follows a standardized approach for building web services that are scalable, flexible, and easy to maintain [1].

3.3 Django REST framework

To create REST API with Django, I installed the Django REST framework package, available from [7]. The framework comes with much built-in functionality, including authentication, serialization, and browsable API, which generates human-friendly output for each resource of API. It improves the usability of REST API for developers with customizable frontend and browsing. The browsable API allows the creation of resources by submitting an HTML form instead of JSON format.

3.4 Celery

Executing tasks, such as video processing, report generation, or sending emails, within the primary application process can result in the

application becoming unresponsive to client requests. In order to prevent this behavior and maintain accessibility and responsiveness to client requests, these tasks should be executed in the background. This approach enables the application to execute numerous tasks simultaneously without impacting the performance of the main app process.

Flower and Celery are Python libraries commonly used in Django projects for background task processing.

Celery is a distributed task queue that enables asynchronous task execution across multiple workers. Task queues are used to distribute work across threads or machines. Celery communicates via messages, usually using a broker to mediate between clients and workers [8]. In this context, a client can be any process or system, such as a Django web application.

Celery documentation recommends multiple broker transport alternatives, for example, RabbitMQ, Redis, Amazon SQS, or Zookeeper. I used Redis because it is easy to set up in Django. Also, Redis is an in-memory key/value data store and can be used as a database, cache, or message broker [9].

To initiate a task, the client adds a message to the queue, and the broker then delivers that message to a worker.

Flower is a web-based monitoring and administration tool for Celery. It provides a user-friendly dashboard for viewing the status of task queues, inspecting individual tasks, and monitoring Celery workers in real time.

4 Building a REST API

The final result of the application is a functional system that simulates matches and events in football leagues by generating randomized data. The results of matches are randomly generated, favoring the home team, which has a slightly higher probability of scoring more goals in that particular match than the away team. The system generates more random data but accurately and precisely to a football match – team statistics, player statistics, match summary and commentary.

The implementation covers only the backend site, meaning it does not support any frontend-services. Although, it uses built-in static files provided by the Django REST framework and its browsable API. The primary focus is developing a robust REST API with a relational database.

A website user can create an account, log in and choose their favorite leagues and teams. After the creation of the account, a user receives a confirmation email, as confirmation, that their account was successfully created.

Also, users receive notifications in the form of an email about the latest results of their favorite teams. The email contains a short message with the result of that match. Once per week, users receive another informative email, which is an update of standings in all their favorite leagues.

The use case diagram 4.1 summarizes the system's behavior.

4.1 Use Case Diagram

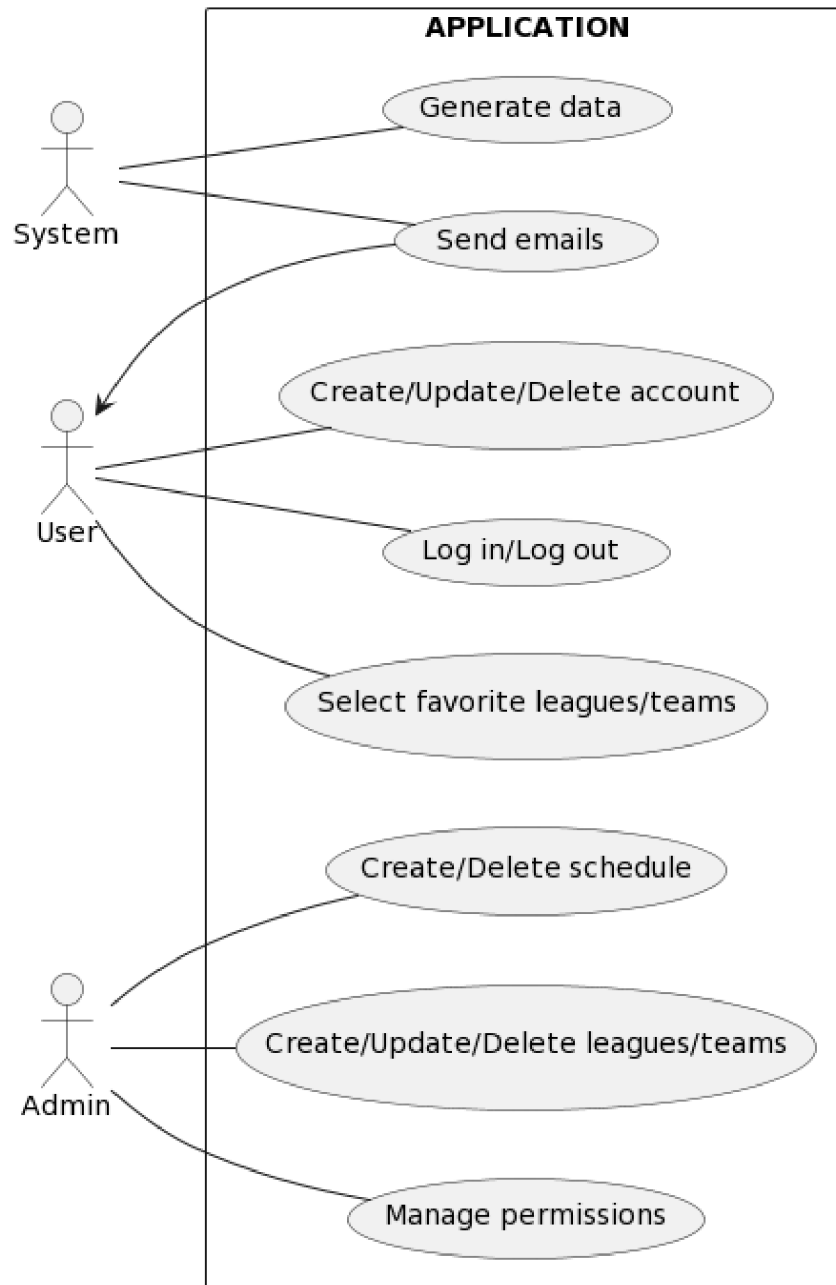


Figure 4.1: Use Case Diagram

4.2 Dependencies

To avoid security and compatibility issues, creating a virtual environment for a Python project is a best practice to keep third-party libraries away from the operating system. There are many tools how for handling virtual environments. Part of the Python standard library is `venv`, which adds no extra functionality except for creating a virtual environment.

I keep all installed dependencies by myself and their extra dependencies separately because it usually results in a complex recursive tree of all project dependencies. Therefore, I created one file for keeping dependencies that I installed in one file (`requirements.in`), and then all transitive dependencies, including their versions, contain a standard `requirements.txt` file.

4.3 ERD diagram

Based on my models, I generated an ERD diagram using a Django app called `django-extensions`, which allows to create a class diagram from models of specified apps and return it as an image. The diagram holds relations among all classes as well as their alphabetically sorted fields. The output is shown in Figure 4.2.

Generally, the URL address does not contain an id of the objects as a URL slug to make it easier for search engines to find content. I included a slug field into many classes, which is auto-generated from other attributes before saving the object in the database.



4.4 Serializers

Serializer converts data models and complex queries from the database into types understandable by frontend frameworks, such as JSON or XML. It is conventional in Django to place serializer classes in a separate Python module called `serializers.py`.

I used the `ModelSerializer` and `HyperlinkedModelSerializer` classes available in the Django REST framework [7].

`HyperlinkedModelSerializer` is the same class as `ModelSerializer`, except it includes the `url` field of the object, which is handy when browsing in the REST API.

These classes are closely related to the model definitions and provide a simplified way to define serializers and avoid the redundancy of defining all attributes of a model. They include predefined validation, representation, and other methods to avoid duplication across all serializer classes. These methods can be overwritten to custom implementation.

Changing the default value of the `lookup_field` keyword in these classes is necessary to reflect the slug field in the URLs.

```
class League(models.Model):
    name = models.CharField(max_length=50)
    slug = models.SlugField(unique=True)
    country = CountryField()
    season = models.PositiveSmallIntegerField()
    ...

class LeagueSerializer(
    NestedHyperlinkedModelSerializer
):
    class Meta:
        model = League
        fields = [
            "url",
            "name",
            "country",
            "season",
            "results",
            "fixtures",
```

```
        "standings",
    ]
    extra_kwargs = {
        "url": {
            "lookup_field": "slug"
        }
    }

    country = CountryField(
        name_only=True
    )
    results = NestedHyperlinkedIdentityField(
        read_only=True,
        view_name="results-list",
        lookup_url_kwarg="league_slug",
        parent_lookup_kwargs={
            "league_slug": "slug"
        },
    )
    fixtures = NestedHyperlinkedIdentityField(
        read_only=True,
        view_name="fixtures-list",
        lookup_url_kwarg="league_slug",
        parent_lookup_kwargs={
            "league_slug": "slug"
        },
    )
    standings = NestedHyperlinkedIdentityField(
        read_only=True,
        view_name="standings-list",
        lookup_url_kwarg="league_slug",
        parent_lookup_kwargs={
            "league_slug": "slug"
        },
    )
```

In the example above, `LeagueSerializer` is a subclass of `NestedHyperlinkedModelSerializer`, which extends the function-

ality of `HyperlinkedModelSerializer` and allows the addition of hyperlinks of nested resources.

Nested resources are used when one resource is contained/nested within another resource. The URL structure can represent this relationship by including the parent and child resources in the URL path. The desired URL signature looks like this:

```
/parent/{parent_pk}/child/{child_pk}
```

Using nested resources in REST APIs can help to organize resources and provide a clear URL structure that reflects the relationships between resources. Nested serializer classes are part of the `drf-nested-routers` package [10]. Nested routers are explained in the section below 4.6.

4.5 View sets

To make a view function appear in browsable API from the Django REST framework, it needs to have `@api_view()` decorator, which takes a list of allowed HTTP methods as an argument.

```
from rest_framework.request import Request
from rest_framework.response import Response

@api_view(["GET", "POST"])
def league_list(request: Request) -> Response:
    if request.method == "GET":
        queryset = ...
        serializer = ...
        return Response(serializer.data)
    elif request.method == "POST":
        queryset = ...
        serializer = ...
        return Response(serializer.data)
```

Django REST framework supports function-based views as well as class-based views. Often, HTTP methods, represented by these views functions, use similar logic when dealing with different resources or models. Each method has a similar query and then returns data from

this query passed to a serializer class. To avoid duplication across these functions, I used class-based views, which combine the same logic of multiple related views in one class. The example below is a class-based view, which inherits from `APIView` parent class. It reduces some duplication by hiding multiple if statements.

```
from rest_framework.views import APIView

class LeagueList(APIView):
    def get(
        self, request: Request
    ) -> Response:
        ...

    def post(
        self, request: Request
    ) -> Response:
        ...
```

To simplify class-based views even more, Django supports so-called view sets. A view set class inherits from mixin classes, which provide action methods similar to HTTP methods.

- `CreateModelMixin` – create model instance
- `RetrieveModelMixin` – retrieve model instance
- `UpdateModelMixin` – update model instance
- `DestroyModelMixin` – destroy model instance
- `ListModelMixin` – list model instances
- `GenericViewSet`¹

These five mixins and the `GenericViewSet` class compose together `ModelViewSet`, which supports executing all HTTP methods. Any combination of these mixins is applicable. For example, `RetrieveModelMixin` and `ListModelMixin` compose `ReadOnlyViewSet`.

1. The `GenericViewSet` class does not provide any actions by default but does include the base set of generic view behavior, such as the `get_object` and `get_queryset` methods.

The view set class must contain at least two attributes: `queryset` for querying the objects from the database and `serializer_class` for serializing these objects. Methods (`get_query_set`, `get_serializer_class`) can overwrite these attributes if, for example, the `queryset` attribute requires specific logic for different HTTP methods handlers.

```
class LeaguesViewSet(ReadOnlyModelViewSet):  
    lookup_field = "slug"  
    queryset = League.objects.all()  
    serializer_class = LeagueSerializer
```

These view sets make a significant level of abstraction compared to view functions, but they confirm one of Django's principles, which encourages rapid development and clean, pragmatic design.

4.6 Routers

Automated registration of views to URL patterns can be achieved through a router. The process of registering view sets with the router is highly abstract and simplifies the overall implementation of the API. The URL configuration of the app contains the URL patterns of router URLs.

However, when using nested resources, a nested router is required. Since the Django REST framework does not provide a nested router, I had to use the same package I used to add hyperlinks of nested resources [10].

The appearance of the browsable REST API is illustrated in the Figure 4.3.

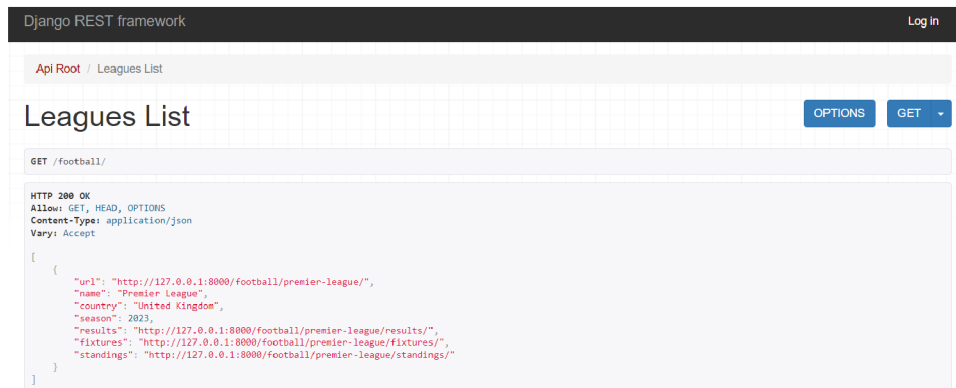


Figure 4.3: Django REST framework browsable API

4.7 Background tasks

Whenever an admin user generates matches via the admin interface, the system schedules asynchronous tasks at a fixed time. It generates data for each match as a background task 90 minutes after the fixed match start and then sends an email, with the result, to all users that mark one of the teams as their favorite. The match is marked as active in the REST API during these 90 minutes.

The system has one more scheduled periodic task for notifying users about the latest standings in their favorite leagues. Figure 4.4 shows the profile endpoint, which allows marking the user's favorite teams and leagues.

4. BUILDING A REST API

Django REST framework admin

Api Root / Profile

Profile

DELETE GET

GET /profile/

HTTP 200 OK
Allow: GET, PUT, DELETE
Content-Type: application/json
Vary: Accept

```
{
  "username": "admin",
  "email": "admin@test.com",
  "my_leagues": [],
  "my_teams": []
}
```

Raw data HTML form

Username
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email

My leagues

My teams

PUT

Figure 4.4: REST API profile endpoint

4.8 Signals

Django offers a set of pre-built signals that enable user-defined code to receive notifications for specific events within the framework. These signals are implemented at the model level and are triggered during various stages of the model's lifecycle. Examples of commonly used signals:

- pre-save
- post-save
- pre-delete

- post-delete

Functions or apps can listen to these signals and perform other actions after they receive a signal. The responsibilities are better distributed across multiple blocks of code. The functions that listen to particular signals are called signal handlers.

Modules that define signals and signal handlers must be imported by overriding the `ready()` method in the `app.py` module in the configuration class of the app.

I implemented a signal handler, which listens to a `post_save` signal of the `User` model. A newly created `User` object fires a `post_save` signal whenever a website user creates an account. The following code snippet is a signal handler that listens to the `post_save` signal from the `User` model and sends a confirmation email to a new user that their account was successfully created.

Figure 4.5 shows the registration endpoint in the REST API.

Django REST framework Log in

Api Root / Register

Register

GET /register/

HTTP 405 Method Not Allowed
Allow: POST
Content-Type: application/json
Vary: Accept

```
{  
  "detail": "Method \"GET\" not allowed."  
}
```

[Raw data](#) [HTML form](#)

Username
Required. 150 characters or fewer. Letters, digits and @/+/+/_ only.

Password

Email

Figure 4.5: REST API register endpoint

5 Testing and Optimization

This chapter explains and demonstrates the REST API testing process on multiple levels.

5.1 Automated Testing

Automated testing can be applied to different types of testing, including unit, integration, system, or regression testing.

Python offers, unsurprisingly, multiple test frameworks. I used the `pytest` test framework for unit and integration testing because it allows writing tests with a few lines of code and has a plugin for testing Django applications.

When writing unit tests, I tried to follow the so-called AAA pattern, a standard to divide a test among three sections – Arrange, Act, and Assert.

In designing the tests, I focused on testing the system's behavior, specifically how API endpoints behave, rather than test individual API components, like models, serializers, or views. This approach ensures flexibility of the tests, even if the implementation of these components changes in the future.

5.1.1 Code quality

Usually, part of the testing process in Python projects is ensuring that the code conforms to specific coding guidelines. Running a separate command for each static code analyzer is a time-consuming process. In that situation, a command line tool, `tox`, could be a helpful solution.

`tox` is used for managing, creating, and running test environments in Python projects. All test environments are invoked within a single command and executed inside an isolated environment.

5.2 Performance Testing

Performance testing is another crucial part of testing software quality. In the context of REST API, performance testing is usually done by sending HTTP requests to various endpoints and determining which

are slower and take too much time to retrieve data from the database and the limit in terms of API users.

A special type of performance testing is so-called stress testing. Stress testing is similar to basic performance testing, except the number of users and their requests is much higher to identify the limits of API users and when the system starts to fail. Stress testing should be done against the production server because the development server is usually very slow.

For a demo performance test, I used a performance testing tool for Python called Locust. It is unrelated to any specific framework. Figure 5.1 shows the performance test.



The screenshot shows the Locust web interface with a dark green header and a table of statistics. The table has columns for Type, Name, # Requests, # Fails, Median (ms), 90%ile (ms), 99%ile (ms), Average (ms), Min (ms), and Max (ms). The data is as follows:

Type	Name	# Requests	# Fails	Median (ms)	90%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
GET	/football	61	0	7500	29000	34000	13290	150	33709
GET	/football/premier-league	46	0	3900	29000	32000	10579	131	31896
GET	/football/premier-league/fixtures	69	0	5200	13000	15000	6408	194	14857
GET	/football/premier-league/results	63	0	7800	13000	16000	6745	179	16114
GET	/football/premier-league/standings	66	0	5200	13000	16000	6213	178	15732
Aggregated		305	0	5200	23000	32000	8441	131	33709

Figure 5.1: Locust performance test

Analyzing project performance is called profiling. For profiling API endpoints, I used Silk, a live profiling and inspection tool primarily used for the Django framework. Silk intercepts each request in the API and collects data displayed on the Silk dashboard at the localhost web address [11]. Figure 5.2 shows the Silk dashboard and collected information while processing requests from Locust. The response times are prolonged because of the overflow added to processing requests by Silk.

Summary	Requests	Profiling	Clear DB	View: Rows	Show: 25	Order: Recent	Order: Descending
11:46:06.892	200 GET	/football/premier-league/results/	532ms overall	28ms on queries	2 queries		
11:46:06.507	200 GET	/football/premier-league/fixtures/	541ms overall	25ms on queries	2 queries		
11:46:06.332	200 GET	/football/	10594ms overall	31ms on queries	1 queries		
11:46:06.330	200 GET	/football/premier-league/fixtures/	926ms overall	24ms on queries	2 queries		
11:46:06.238	200 GET	/football/premier-league/	10531ms overall	34ms on queries	1 queries		
11:46:05.814	200 GET	/football/premier-league/results/	693ms overall	32ms on queries	2 queries		
11:46:05.507	200 GET	/football/premier-league/standings/	200ms overall	42ms on queries	2 queries		
11:46:05.324	200 GET	/football/premier-league/results/	645ms overall	42ms on queries	2 queries		
11:46:05.303	200 GET	/football/premier-league/fixtures/	647ms overall	50ms on queries	2 queries		
11:46:05.054	200 GET	/football/	12063ms overall	48ms on queries	1 queries		
11:46:04.788	200 GET	/football/premier-league/results/	642ms overall	39ms on queries	2 queries		
11:46:04.355	200 GET	/football/premier-league/results/	603ms overall	54ms on queries	2 queries		

Figure 5.2: Silk profiling dashboard

5.2.1 Optimization

Optimizing database queries is a commonly recommended technique for improving performance. Various optimizations can be applied to the queries to achieve faster response times, such as rewriting queries with SQL, redesigning the database, or caching.

One of the common factors that can cause an API endpoint to slow down is inefficient querying or accessing the database. The Django Debug Toolbar is a tool that provides developers with detailed information on the performance and behavior of the Django applications. It is a tool that primarily monitors the database. In a browser, a sidebar panel shows actions needed to display the current page, including executed SQL queries. It helps to determine which queries should be optimized or rewritten.

Django documentation also has database access optimization document¹, which is a summary of all possible query set methods to fasten the database queries.

Another option for improving the performance of the API is to cache data coming from the database. Django has a built-in caching framework as well. The cache framework supports several caching

1. Database access optimization – <https://docs.djangoproject.com/en/4.1/topics/db/optimization/>

backends, including in-memory caching, file-based caching, database caching, and caching with external services like Memcached or Redis. Since I used Redis as a message broker to communicate with Celery workers, I also used Redis as a cache backend.

Conclusion

The thesis comprehensively explores integrating the Django web framework with RESTful API development. It includes a practical demonstration of developing REST APIs using Django and serves as a resource for readers seeking to leverage Django for their backend development requirements. The Swagger/OpenAPI 2.0 specification is part of the API at the `/swagger` endpoint.

The application can be run as a Docker container. One of the application's source files is a script `wait-for-it.sh`, which waits on the availability of a host and TCP port. *It is useful for synchronizing the spin-up of interdependent services, such as linked docker containers* [12].

A topic for future work might be adding frontend services or extending the API with additional data.

Bibliography

1. RED HAT. *What is a REST API?* [online]. 2020. [visited on 2023-05-01]. Available from: <https://www.redhat.com/en/topics/api/what-is-a-rest-api>.
2. ARJANCODES. *Clean Architecture vs Django's Pragmatic Design* [online]. 2023. [visited on 2023-05-01]. Available from: <https://www.youtube.com/shorts/nHd7aYUQMdY>.
3. JETBRAINS. *Python Developers Survey 2021 Results* [online]. 2021. [visited on 2023-05-01]. Available from: <https://lp.jetbrains.com/python-developers-survey-2021>.
4. *Django documentation* [online]. 2023. [visited on 2023-05-01]. Available from: <https://docs.djangoproject.com/en/4.2>.
5. *Flask documentation* [online]. 2023. [visited on 2023-05-01]. Available from: <https://flask.palletsprojects.com/en/2.2.x>.
6. RAMÍREZ, Sebastián. *FastAPI documentation* [online]. 2023. [visited on 2023-05-01]. Available from: <https://github.com/tiangolo/fastapi>.
7. *Django REST framework* [online]. 2023. [visited on 2023-05-01]. Available from: <https://www.django-rest-framework.org>.
8. *Celery: Distributed Task Queue* [online]. 2023. [visited on 2023-05-01]. Available from: <https://github.com/celery/celery#whats-a-task-queue>.
9. *Backends and Brokers – Celery documentation* [online]. 2023. [visited on 2023-05-01]. Available from: <https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/index.html>.
10. SILVA, Alan Justino da. *DRF Nested Routers* [online]. 2023. [visited on 2023-05-01]. Available from: <https://github.com/alanjds/drf-nested-routers>.
11. GULEA, Iulian. *Guide to Django Performance Testing and Optimization* [online]. 2023. [visited on 2023-05-01]. Available from: <https://www.toptal.com/python/performance-optimization-testing-django>.

BIBLIOGRAPHY

12. HALL, Giles. *Wait for it* [online]. 2023. [visited on 2023-05-01]. Available from: <https://github.com/vishnubob/wait-for-it>.

A An appendix

Part of the thesis is a zip file, which contains source files of the entire project.